Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 03 Notes
I/O + Command Line + Types + Operators

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 01) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 Console I/O: `Scanner` (most common for beginners) . . . 2
5.2 Console I/O: `BufferedReader` (fast, line-oriented) . . . 3
5.3 Output formatting . . . 3
5.4 Command line arguments . . . 3
5.5 Data types, operators, precedence, casting . . . 3
5.5.1 Primitive data types (quick) . . . 3
5.5.2 Operator pitfalls you must know . . . 3
5.5.3 Casting . . . 4

6 Worked Example: Args + Scanner + Formatting **4**

7 In-Class Exercises (with brief solutions) **4**

8 Self-Study Practice (no solutions) **5**

9 Checkpoint Questions (with sample answers) **5**

1 Lecture Overview

This lecture focuses on the day-to-day basics you need to write useful Java programs:

- taking input (keyboard / terminal),
- printing output in a readable way,

- using command line arguments,
- understanding primitive data types and operators (and common pitfalls).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 27, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 27, 2026).

2 Syllabus Alignment (Unit 01)

Unit 01: I/O statements, command line arguments, data types, variables, operators

3 Learning Outcomes

After this lecture, you should be able to:

- Read console input using `Scanner` and (at a basic level) `BufferedReader`.
- Use `System.out.println` and `System.out.printf` for formatted output.
- Use `String[] args` to accept command line inputs and parse simple values.
- Choose correct primitive types and apply operators with correct precedence and casting.

4 Key Terms (Quick)

- **Token vs line**: token = one word/number; line = full line of text.
- **Formatting**: printing numbers/text with a chosen format (decimal places, alignment).
- **Primitive types**: built-in value types like `int`, `double`, `char`, `boolean`.
- **Type casting**: converting a value from one type to another (widening/narrowing).
- **Precedence**: order in which operators are evaluated.

5 Detailed Notes

5.1 Console I/O: `Scanner` (most common for beginners)

`Scanner` reads tokens from an input source like `System.in`.

- Use `nextInt()`, `nextDouble()`, `next()` (word), `nextLine()` (whole line).
- **Common pitfall:** mixing `nextInt()` and `nextLine()`.

  After reading a number, the newline remains in the buffer; the next `nextLine()` may return an empty string.

  Fix: call an extra `nextLine()` after reading the number.

5.2 Console I/O: `BufferedReader` (fast, line-oriented)

`BufferedReader` reads full lines as `String`, and you parse manually:

- Good when you want complete control or faster I/O.
- Slightly more code than `Scanner` (because you must parse numbers using `Integer.parseInt`, etc.).

5.3 Output formatting

Use:

- `System.out.println(...)` for simple output.
- `System.out.printf("x = %d, y = %.2f%n", x, y);` for formatted output.

Common format specifiers:

- `%d` integer, `%f` floating-point, `%s` string, `%n` newline.

5.4 Command line arguments

When you run a Java program, anything after the class name becomes a string in `args`.

- Example: `java Sum 10 20 30`
- Then: `args[0] = "10"`, `args[1] = "20"`, etc.

You often parse numbers using:

- `Integer.parseInt(args[i])`
- `Double.parseDouble(args[i])`

Pitfall: if input is not numeric, parsing throws `NumberFormatException`.

5.5 Data types, operators, precedence, casting

5.5.1 Primitive data types (quick)

- Integer types: `byte`, `short`, `int`, `long`
- Floating types: `float`, `double`
- Other: `char` (Unicode), `boolean`

5.5.2 Operator pitfalls you must know

- **Integer division:** `5/2 == 2` (not 2.5). Use `5/2.0` or cast: `(double)5/2`.
- **Overflow:** `int` can overflow silently.

  Example: `int x = 2_000_000_000; x = x + 1_000_000_000;` (wraps around).

- **Short-circuit logic:** `&&` and `||` may skip evaluating the right side.

  This is useful for null checks: `s != null && s.length() > 0`.

- **Precedence:** multiplication happens before addition. Use parentheses when teaching/learning.

5.5.3 Casting

Widening (safe) happens automatically: `int` → `double`. **Narrowing** may lose data and requires explicit cast: `double` → `int`.

6 Worked Example: Args + Scanner + Formatting

```java
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        int maxMarks = 100;
        if (args.length > 0) maxMarks = Integer.parseInt(args[0]);

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter marks (0-" + maxMarks + "): ");
        int marks = sc.nextInt();

        double percent = marks * 100.0 / maxMarks;
        System.out.printf("Hello %s, marks=%d/%d, percent=%.2f%%%n",
                name, marks, maxMarks, percent);
        sc.close();
    }
}
```

What to observe:

- How `args` is always a string array and needs parsing.
- How we used a default value (100) and optionally override it using `args[0]`.
- How formatted output (`printf`) improves readability.
- Why we used `100.0` in the formula (to avoid integer division mistakes).

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 27, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 27, 2026).

7 In-Class Exercises (with brief solutions)

1. What is the output of `System.out.println(5/2);`? **Answer:** `2`.
2. How to get `2.5` from 5 and 2? **Answer:** `5/2.0` or `(double)5/2`.

3. If you run `java Demo A B`, what is `args.length`? **Answer:** 2.

8 Self-Study Practice (no solutions)

1. Write a program `SumArgs` that sums integer command line arguments.
2. Write a program that reads a line and prints: (a) length, (b) first character, (c) uppercase version.
3. Create 5 expressions and predict results using precedence; then verify by running Java.

9 Checkpoint Questions (with sample answers)

Q1. Why is `Scanner.nextLine()` tricky after `nextInt()`?

Sample answer: `nextInt()` leaves the newline in the input buffer, so the next `nextLine()` may read the leftover newline and return an empty string. We can consume the newline by calling `nextLine()` once after reading the number.

Q2. Why is `5/2` not 2.5 in Java?

Sample answer: Both operands are integers, so Java performs integer division and discards the fractional part. If either operand is `double`, it performs floating-point division.